

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Portabilidad y plataformas

Francisco Javier Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** las cuatro formas de portabilidad y su independencia.
- **Identificar** supuestos de plataforma implícitos en el código.
- **Explicar** por qué las diferencias de plataforma son fuente de fallos de corrección y de seguridad.
- **Aplicar** una plantilla de especificación de plataforma para agentes.
- **Reconocer** que la portabilidad de seguridad no se hereda automáticamente.

Cuatro formas de portabilidad

De fuente. El programa se compila/interpreta en la nueva plataforma sin cambios.

Binaria. El mismo ejecutable se ejecuta (requiere compatibilidad de ABI).

De comportamiento. Produce resultados *observacionalmente equivalentes*.

De seguridad. Conserva sus propiedades de seguridad bajo el mismo modelo de amenaza en la nueva plataforma.

Las cuatro son **independientes**: un programa puede ser portable de fuente y no de comportamiento; portable de comportamiento y no de seguridad.

Por qué importa con IA

- Los asistentes codifican supuestos de plataforma implícitos: separador de ruta, sensibilidad a mayúsculas, tamaño de `long`.
- Esos supuestos no aparecen en el *prompt* ni en las pruebas.
- La portabilidad es una propiedad *relacional* entre un programa y un conjunto *declarado* de plataformas.

Las diferencias de plataforma son fuente directa de fallos de corrección y seguridad; la portabilidad no es neutral respecto de la seguridad.

Sistemas operativos, API y ABI

Dos familias que se parecen en la superficie y difieren en los detalles que deciden la corrección:

- **POSIX** (Linux, macOS, BSD): `open`, `read`, `write`, `stat`, `fork`, `exec`, señales.
- **Win32/Win64**: `CreateFile`, `ReadFile`, `CreateProcess`, objetos con *handles* y ACL.

Deshacer dos simplificaciones: «Linux» no es una plataforma única (núcleo, *glibc*/*musl*, distribución, arquitectura) y «Windows» no es «POSIX con otra barra» (procesos, permisos y caracteres son distintos).

Diferencias POSIX frente a Win32

Aspecto	POSIX	Win32/Win64
Separador de ruta	/	\
Sensibilidad a mayúsculas	Sensible (típico)	Insensible (típico)
Modelo de permisos	Bits `rwx`, `umask`	ACL, tokens, SID
Bit ejecutable	Sí, en el modo	No; por extensión y ACL
Enlaces simbólicos	*symlink* nativo	*symlink*/*junction* (privilegio)
Descriptor de fichero	Entero	*HANDLE* opaco; *fd* emulado
Fin de línea en modo texto	No traduce	CRLF ↔ LF
Modelo de procesos	`fork`/`exec`, señales	`CreateProcess`
Nombres reservados	No	`CON`, `NUL`, `COM1-9`
Longitud de ruta	`PATH_MAX`	`MAX_PATH` (260)
Modelo de datos 64-bit	LP64 (`long` 64)	LLP64 (`long` 32)
`wchar_t`	32 bits (UTF-32)	16 bits (UTF-16)

La portabilidad no es neutral para la seguridad

- La insensibilidad a mayúsculas, los nombres de dispositivo reservados, el seguimiento de enlaces o la ausencia de `O_NOFOLLOW` pueden explotarse.
- Un control que se apoya en una suposición de plataforma («el prefijo identifica el directorio», «este nombre no existe», «no es un enlace») deja de ser un control cuando la suposición no se cumple.

Los detalles dependen de la versión y deben verificarse contra la documentación vigente (POSIX.1-2017 [1], Win32 [2]).

Rutas, separadores y canonicidad

- Una ruta es una cadena, pero el mismo objeto puede escribirse con separadores, mayúsculas, `./..`, enlaces o rutas relativas distintas.
- La forma única se obtiene por *canonicalización* (`realpath`, `Path.resolve()`).
- El mismo control de confinamiento puede ser correcto en Linux e insuficiente en Windows si se apoya en comparaciones de cadenas que ignoran la plataforma.

Windows normaliza nombres: los que terminan en punto o espacio pueden ser equivalentes; `CON`, `NUL`, `COM1-9` designan dispositivos. Un validador que solo rechace `..` y `/` no cubre estos casos.

Ruta con separador POSIX (frágil en Windows)

```c
/* Material didactico. Solo en entorno aislado y autorizado. */
const char *nombre_base(const char *ruta)
{
    const char *base = strrchr(ruta, '/'); /* en Windows puede usar '\' */
    return base ? base + 1 : ruta;
}
```

Listing 1: Asume el separador '/' al construir y analizar la ruta.

Si `nombre_base` se usa para *validar* el nombre de salida y devuelve la ruta completa (no encontró `/`), la validación **no se aplica**. Corrección: delegar en la plataforma (`os.path.join`/`basename`, `std::filesystem`).

Patrón portable contextual de ruta de salida

```python
import os
def ruta_de_salida(dir_salida, nombre):
    base = os.path.basename(nombre)          # descarta componentes de ruta
    if not base or base in (".", ".."):
        raise ValueError("nombre de salida no valido")
    return os.path.join(dir_salida, base)    # separador nativo
```

Listing 2: Correcto respecto de las precondiciones declaradas; NO universalmente seguro.

Enlaces simbólicos y permisos

- En POSIX, un enlace simbólico apunta a otro objeto; las llamadas lo siguen por defecto y `O_NOFOLLOW` impide seguir el último componente.
- En Windows hay enlaces simbólicos y *junctions* con otra semántica y requisitos de privilegio; `O_NOFOLLOW` no tiene equivalente directo.
- El modelo de permisos es más divergente: bits `rwx` y `umask` frente a ACL y tokens. No hay bit ejecutable equivalente en Windows.

Un programa que «protege» un temporal con 0600 protege en POSIX, pero no expresa la misma garantía en Windows, donde deciden las ACL.

`O_NOFOLLOW`: control POSIX sin equivalente en Win32

```c
int abrir_sin_seguir_enlace(const char *ruta)
{
    return open(ruta, O_RDONLY | O_NOFOLLOW | O_CLOEXEC);
}
```

Listing 3: Impide seguir un enlace simbólico final en POSIX; no portable a Win32.

Incluso en POSIX es contextual: protege el último componente, no los intermedios, y no elimina la ventana de carrera si cambia el directorio entre comprobación y apertura.

Descriptores de fichero y *handles*

- POSIX: un *file descriptor* es un entero pequeño; se hereda a través de `fork` y, salvo `O_CLOEXEC`, de `exec` (fuente de fugas).
- Windows: el recurso es un *HANDLE* opaco; la CRT ofrece un *fd* emulado cuya semántica no coincide con POSIX en todos los casos.
- Asumir que los descriptores son enteros consecutivos o que el orden de cierre importa igual es frágil y no portable.

Suposición del valor del descriptor (frágil)

```c
int fd = open("datos.csv", O_RDONLY);
if (fd < 0) { return 1; }
printf("descriptor esperado: %d\n", fd); /* se asume fd == 3 */
close(fd);                                /* no se comprueba (CWE-252) */
```

Listing 4: Asume que el descriptor es un entero predecible y no comprueba `close`.

Corrección: tratar el descriptor como valor opaco y verificar cada operación.

Temporales, fin de línea y codificación

- Temporales: `mkstemp` (POSIX) frente a `_mktemp_s`, `GetTempFileName` (CRT); directorio `TMPDIR` frente a `TEMP`/`TMP`.
- Fin de línea: POSIX usa LF; Windows CRLF. En modo texto, la CRT de Windows traduce al leer y escribir; en POSIX no. Contar bytes o calcular desplazamientos da valores distintos.
- Regla práctica: fijar codificación y fin de línea *en el código* (`encoding="utf-8"`, `newline=""`), no en la plataforma.

Conteo de bytes dependiente del modo y la plataforma

```c
long tamano_logico(const char *ruta)
{
    FILE *f = fopen(ruta, "r");   /* modo texto: traduccion dependiente */
    if (!f) return -1;
    long n = 0; int c;
    while ((c = fgetc(f)) != EOF) n++;
    if (ferror(f)) { fclose(f); return -1; }
    fclose(f);
    return n;   /* en Windows con CRLF, n es menor que en binario */
}
```

Listing 5: En modo texto, Windows traduce CRLF<->LF; el recuento cambia.

Unicode y modelos de caracteres

- `wchar_t`: 16 bits en Windows (UTF-16), típicamente 32 en Linux/glibc (UTF-32). Asumir 2 o 4 bytes calcula mal los tamaños.
- Python 3 indexa por *code point*; Java por unidad UTF-16; Rust `String` es UTF-8 y `len()` cuenta bytes.
- Distinguir *bytes*, *code units* y *code points* es imprescindible.
- Siguen existiendo diferencias de normalización (NFC/NFD) y de BOM.

La codificación no basta: hay que fijar también la normalización y el tratamiento del BOM; un CSV con BOM puede leerse de forma distinta según la plataforma.

Tamaño de `wchar_t` (correcto con `sizeof`)

```c
wchar_t *duplicar_ancho(const wchar_t *s)
{
    size_t n = wcslen(s);
    wchar_t *r = malloc((n + 1) * sizeof(wchar_t)); /* no literal 2/4 */
    if (!r) return NULL;
    wcscpy(r, s);
    return r;
}
```

Listing 6: Correcto porque usa sizeof; falla si se sustituye por el literal 2 o 4.

Procesos y concurrencia

- POSIX crea procesos con `fork` y los sustituye con `exec`; Windows con `CreateProcess` (sin `fork`). Las señales no tienen equivalente exacto en los eventos de consola.
- Las carreras de datos (CWE-362) existen en ambas familias, pero el modelo de memoria y las garantías de visibilidad difieren.
- Que una carrera «no se manifieste» en las pruebas no significa que no exista (suficiencia finita).

Un comando construido por interpolación (`os.system("rm -rf " + d)`) no es portable y además es inyección de comandos (CWE-78). Corrección: API con lista de argumentos (`shutil.rmtree`, `subprocess.run([...], shell=False)`).

Compiladores, comportamiento indefinido y arquitectura

El estándar de C distingue tres tipos de comportamiento:

Definido por la implementación. El estándar permite varias opciones y exige documentar la elegida (p. ej. signo de `char`, ancho de `int`/`long`).

No especificado. No fija una única opción ni exige documentarla (orden de evaluación).

Indefinido (UB). No impone significado: desbordamiento con signo, acceso fuera de límites, lectura no inicializada, violación de *strict aliasing*.

El compilador puede asumir que el UB no ocurre y optimizar en consecuencia: el programa «funciona» con una configuración y falla con otra. Las reglas CERT [5] tratan de eliminar estas construcciones.

Modelo de datos LP64 frente a LLP64

```c
long bytes_totales(long elementos, long tamano)
{
    return elementos * tamano;  /* posible CWE-190 */
}
```

Listing 7: LP64: long=64 (Linux); LLP64: long=32 (Windows). Un cálculo que asume 64 puede desbordar.

La arquitectura añade ancho de palabra, alineamiento y *endianness*.

Signo de `char` (definido por la implementación)

```c
int es_negativo(char c)
{
    return c < 0;   /* puede ser siempre falso segun la plataforma */
}
```

Listing 8: 'char' con signo en x86 típico; sin signo en ARM típico.

Ejemplos: válido aquí, incorrecto o inseguro allí

Ejemplo	Válido en	Consecuencia en la otra
Ruta con `/`	POSIX	Análisis falla con \; validación no aplica
`O_NOFOLLOW`	POSIX	No existe igual; puede seguirse el enlace (CWE-59)
Descriptor entero	Supuesto	Valor/herencia no coinciden; recursos no cerrados
Conteo en modo texto	POSIX	Windows traduce CRLF; recuentos cambian
`wchar_t` literal 2/4	Windows	En glibc es 4; búfer mal dimensionado
Shell interpolado	POSIX `sh`	En Windows no existe; inyección (CWE-78)
`long` 64 bits	LP64	En LLP64 es 32; posible desbordamiento (CWE-190)
Signo de `char`	x86	En ARM suele ser sin signo; comparación falsa

Invariante del capítulo

Que un fragmento funcione en una plataforma **no** demuestra que sea correcto ni seguro en otra, ni siquiera que sea seguro en la suya. La portabilidad de seguridad exige declarar el entorno y verificar el control *en ese entorno*. Superar una batería finita de pruebas no demuestra seguridad.

Plantilla de especificación de plataforma para agentes

Propuesta docente: obliga a enunciar los supuestos que el agente resolvería por su cuenta. Se entrega *antes* de pedir el código y se usa *después* para revisarlo.

Campo	Ejemplo	Fallo que previene
P1 SO y versión	Linux x86-64, núcleo y distro	APIs ausentes o distintas
P2 Arquitectura	x86-64, LP64, little-endian	Ancho de palabra o endianness
P3 Compilador	gcc 13, `-O2 -Wall`	Dependencia de UB o extensión
P4 Lenguaje	C11, Python 3.12	Funciones retiradas o no disponibles
P5 APIs y ABI	POSIX.1-2017; no Win32	Primitivas no trasladables
P6 Bibliotecas	openpyxl, reportlab (fijadas)	Dependencias implícitas/vulnerables
P7 Rutas	separador nativo, canonicidad	Path traversal, nombres reservados
P9 Recursos	256 MiB, 30 s, sin red	Agotamiento, exfiltración
P10 Codificación	UTF-8 sin BOM, LF	Confusión de bytes, fin de línea

Enlace con el caso transversal CSV→PDF/Excel

- Rutas y separadores: construir con `os.path.join` y canonicalizar (CWE-22).
- Temporales: `mkstemp`/`NamedTemporaryFile` restrictivo (CWE-377).
- Enlaces simbólicos: no seguir al abrir; evitar TOCTOU (CWE-59/367).
- Fin de línea: `newline=""` al leer CSV; LF o CRLF explícito.
- Permisos: `0600` en POSIX; ACL equivalentes en Windows (CWE-732/276).
- Codificación: UTF-8 declarado; BOM y normalización decididos.
- Bibliotecas PDF/Excel: fijar versión y comprobar avisos (CWE-1395).

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

Patrón portable contextual de escritura de salida

```python
import os, tempfile
def destino_seguro(dir_salida, nombre):
    base = os.path.basename(nombre)
    destino = os.path.join(dir_salida, base)
    raiz = os.path.realpath(dir_salida)
    destino = os.path.realpath(destino)
    if os.path.commonpath([raiz, destino]) != raiz:
        raise ValueError("salida fuera del directorio permitido")
    return destino
# mkstemp devuelve un descriptor; se escribe con os.fdopen (sin reabrir por
    nombre)
```

Listing 9: Escribe a través del descriptor (sin reabrir el temporal por nombre); contextual.

Síntesis del capítulo

- La portabilidad es relacional y sus cuatro formas son independientes.
- Las diferencias POSIX/Win32 afectan a la seguridad: separadores y canonicidad, mayúsculas, nombres reservados, enlaces, permisos, descriptores, fin de línea y modelo de caracteres.
- El compilador y la arquitectura introducen comportamiento definido por la implementación y UB; un fragmento puede funcionar con una configuración y fallar con otra.
- La plantilla de plataforma convierte supuestos implícitos en requisitos explícitos y verificables (Propuesta docente; alineada con SSDF [3] y ASVS [4]).
- En el caso transversal, rutas, temporales, fin de línea, permisos, codificación y bibliotecas son decisiones de plataforma que determinan la seguridad.

Actividad

1. Diagnostique los listados de ruta y de conteo: plataforma válida, supuesto implícito y consecuencia en la otra.
2. Rellene la plantilla de plataforma para el caso transversal en Linux y Windows; señale tres campos que cambian.
3. Explique por qué `O_NOFOLLOW` no tiene equivalente exacto en Win32 y qué control alternativo propondría.
4. Proponga tres pruebas que detectarían los fallos del cuadro de ejemplos y explique por qué superarlas no da «portabilidad y seguridad universales».

¿Es «portable» sinónimo de «seguro»?

- Un programa que se comporta igual en ambas plataformas, ¿conserva sus propiedades de seguridad?

Referencias I



IEEE / The Open Group.

Ieee std 1003.1-2017 (posix.1-2017): Standard for information technology—portable operating system interface (posix).

Norma (POSIX.1-2017).



Microsoft.

Windows api documentation (win32/win64).

Documentación técnica. Consulta: 2026.



National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.

Referencias II



OWASP Foundation.
Application security verification standard (asvs), 2025.
Guía de referencia (versión estable 5.0.0, 2025).



SEI / CERT Division, Carnegie Mellon University.
Sei cert c coding standard.
Technical report, 2016.
Edición CERT C 2016.